Virtual Academic Advisor

Agentic LLM System for GSU Computer Engineering Graduation Analysis

INF473 Project Report

May 9, 2026

Abstract

This report documents the completed Virtual Academic Advisor project, an agentic LLM-based system that evaluates unstructured Galatasaray University Computer Engineering transcripts against graduation requirements. The implemented system combines a language-model transcript parser, a rule-based academic knowledge base, independent verification agents, and a tool-augmented master reporting agent. The result is a web application that accepts transcript text, normalizes academic records, checks mandatory courses, ECTS, GPA, language, and elective requirements, and produces an advisor-style graduation report with clear missing conditions.

Contents

1 Project Overview

The project addresses a practical academic advising problem: students submit transcripts as unstructured text, while graduation eligibility depends on a structured interpretation of the official Computer Engineering curriculum. The system therefore has three core responsibilities:

1. extract reliable structured data from raw transcript text;
2. compare that data with the GSU Computer Engineering graduation rules;
3. generate a concise, human-readable report explaining whether the student satisfies the requirements and, if not, which conditions are blocking graduation.

The official GSU ECTS page for the Computer Engineering undergraduate program is the curriculum reference used by the project. The page lists the semester curriculum, general graduation rules such as the minimum GPA requirement, foreign-language requirement, and the rule that students below 240 ECTS must complete missing credits with INF electives.[1]

2 Project Motivation

We chose this project because graduation eligibility is a practical and high-impact problem for students. A student may have completed many courses but still be blocked by a missing mandatory course, an elective-group rule, a low GPA, insufficient ECTS, or a foreign-language requirement. Manually checking these conditions is time-consuming and error-prone, especially when transcripts include old course codes, repeated courses, failed courses, or curriculum transition rules.

The project is also a suitable use case for an agentic LLM system. The input is unstructured academic text, which is difficult to process with only static forms, while the final graduation decision must remain deterministic and explainable. Therefore, the project allowed us to combine the language understanding capability of LLMs with rule-based verification. This combination matches the academic advising domain: flexible parsing is useful, but the decision itself must be based on explicit university requirements.

3 System Scope

The implementation is a full-stack application:

- **Backend:** FastAPI service in `backend/main.py`.
- **Agent pipeline:** LLM-backed and rule-aware agent logic in `backend/agent.py`.
- **Knowledge base:** encoded GSU requirements in `backend/gsu_requirements.py`.
- **Persistence:** SQLite database via SQLAlchemy models in `backend/models.py`.
- **Frontend:** React/Vite application under `frontend/src`.
- **Validation assets:** 20 transcript scenarios under `analysis-input-scenarios/`.

The system is an agentic academic-advising architecture. Its agents are not all implemented in the same way: some are LLM-backed agents for language-intensive tasks, and some are rule-verifier agents that execute the curriculum knowledge base through Python. The scope of the application covers the complete advising workflow from raw transcript submission to persistent

[1] https://ects.gsu.edu.tr/tr/program/programmedetails/12

analysis history. A student can submit a plain-text transcript through the frontend, the backend stores and deduplicates the input, the agent pipeline evaluates graduation eligibility, and the result is returned as both structured data and an advisor-style natural-language report. The system also exposes agent-level verdicts, missing conditions, and course-level evidence so that the final decision can be inspected rather than treated as an LLM answer.

4 Agentic Architecture Design

4.1 High-Level Design

The project follows a staged agentic workflow:

1. **Input acquisition.** The frontend accepts pasted transcript text or a `.txt` file.
2. **Input validation.** Client and server validation reject empty submissions and likely multi-transcript inputs.
3. **Persistence and deduplication.** The backend stores the raw transcript and uses a SHA-256 text hash to avoid duplicate transcript rows.
4. **LLM parsing.** `TranscriptParserAgent` transforms the raw transcript into typed JSON.
5. **Independent verification.** Three rule-verifier agents evaluate course completion, ECTS, and broader graduation requirements.
6. **Cross-examination.** Agents add short comments on how their findings relate to other agents' findings.
7. **Master decision.** `MasterAgent` aggregates verdicts; the student is eligible only when all verifier agents pass.
8. **Tool-augmented reporting.** The master report step may call a SQLite history tool to compare the current analysis with prior records.
9. **Report review.** `ReportCriticAgent` polishes the final text and removes machine-like wording.
10. **Presentation.** The frontend displays metrics, missing conditions, agent verdicts, course lists, history, and PDF export.

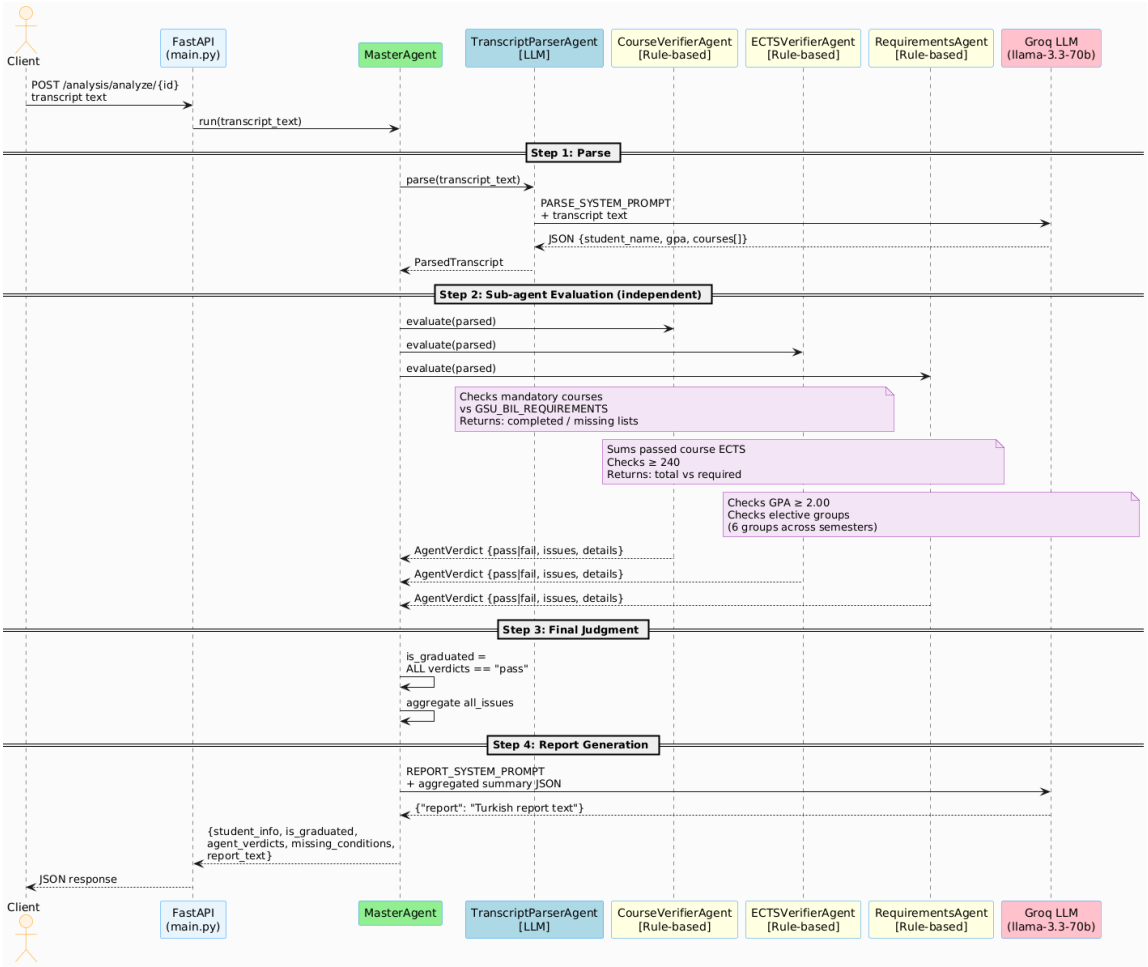

Figure 1: Implemented sequence diagram from the project repository.

5 Component Analysis

5.1 Backend API

The FastAPI backend exposes the following main endpoints:

Method	Endpoint	Purpose
`POST`	`/transcripts/upload`	Validate, hash, and store transcript text.
`POST`	`/analysis/analyze/{id}`	Run the agent pipeline for a transcript.
`GET`	`/analysis/{id}`	Retrieve a stored analysis result.
`GET`	`/analysis/history`	List recent analyses.
`DELETE`	`/analysis/{id}`	Delete an analysis and its transcript.

Table 1: Backend API surface.

The backend also performs lightweight SQLite migration at startup, adding columns such as `text_hash` and `agent_verdicts` for older database files.

5.2 Transcript Validation

`backend/transcript_validation.py` protects the pipeline from common bad inputs. It detects multiple transcripts by checking repeated example headers, conflicting student names, conflicting student numbers, and cumulative ECTS resets in `Genel` rows. The same idea is mirrored in the frontend upload page so the user receives fast feedback before the backend request.

5.3 LLM Client Layer

The LLM integration uses the Groq API. The model cascade is:

- `llama-3.3-70b-versatile`;
- `llama-3.1-8b-instant`;
- `gemma2-9b-it`.

The shared `_chat` wrapper uses temperature `0.0`. When no tools are supplied, it requests JSON-mode output. When tools are supplied, it enables native tool calling and lets the model decide whether to call the tool.

5.4 Knowledge Base

The knowledge base is implemented in `backend/gsu_requirements.py` as a Python dictionary named `GSU_BIL_REQUIREMENTS`. It contains:

- minimum GPA: `2.0`;
- minimum ECTS: `240`;
- language condition: 12 foreign-language credits and English B2+;
- 44 mandatory course entries, including language-course entries;
- 6 elective groups;
- 21 course equivalency mappings;
- transition notes for legacy curriculum cases.

The backend is the source of truth for graduation decisions. The frontend currently displays a hard-coded mandatory-course denominator of 43 in the result card, while the backend requirement list contains 44 entries. This does not change the backend decision, but it is an implementation detail that should be aligned in a future cleanup.

5.5 Data Description

The project data consists of two main sources. The first source is unstructured student transcript text. A transcript contains student identity fields when available, semester headers, course rows, grades, course credits, ECTS values, and cumulative GPA rows. The parser specifically uses course code, course name, grade, and ECTS fields because these values are required for graduation analysis. Failed courses are kept in the parsed output because they affect legacy transition rules and because they must be excluded from passed-ECTS and completed-course calculations.

The second source is the academic rule data derived from the official GSU Computer Engineering curriculum. This data is encoded in the project as mandatory courses, minimum GPA,

minimum ECTS, language requirements, elective groups, old-code equivalencies, and transition notes for legacy curriculum cases. These rules form the structured knowledge base used by the verifier agents.

For evaluation, the repository includes 20 text-based transcript scenarios under `analysis-input-scenarios`. These scenarios cover both successful and unsuccessful graduation cases, including low GPA, insufficient ECTS, missing mandatory courses, missing language requirements, modern elective failures, legacy elective obligations, and old course-code equivalencies. This scenario set was used instead of relying on production student records, which keeps the project easier to test while reducing privacy risk.

5.6 Database and Persistence

The project uses SQLite through SQLAlchemy. The data model contains three tables: `students`, `transcripts`, and `analysis_results`. The database stores raw transcript text, student identity when parsed, GPA, total ECTS, missing courses, missing conditions, agent verdicts, final report text, and timestamps.

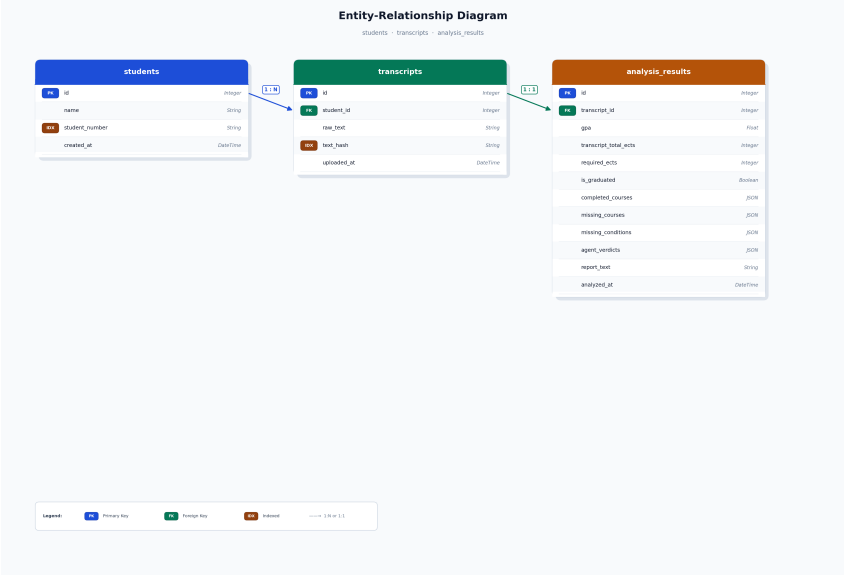

Figure 2: Database schema diagram from the project repository.

5.7 Frontend

The frontend is a React/Vite application with Turkish and English UI strings. It provides:

- transcript paste and `.txt` upload;
- link to the official curriculum page;
- progress steps during analysis;
- result banner with the final report;
- GPA, ECTS, mandatory-course, and missing-condition cards;
- expandable agent-verdict cards;
- completed and missing course lists;
- analysis history with search, filters, sorting, and deletion;
- PDF export through `jsPDF`.

6 Role of Agents

Agent / Role	Input and Output	Responsibility
`TranscriptParserAgent`	Raw transcript text to `ParsedTranscript`.	Uses the LLM parser prompt to extract student identity, cumulative GPA, and all courses, including failed courses. A deterministic override then extracts GPA from the last `Genel` row when available.
`CourseVerifierAgent`	`ParsedTranscript` to `AgentVerdict`.	Checks mandatory course completion against normalized course codes and equivalencies. It strips section suffixes such as `-A` and `-B` before matching.
`ECTSVerifierAgent`	`ParsedTranscript` to `AgentVerdict`.	Sums ECTS only for passed courses and checks the 240 ECTS threshold. Failed grades include `FF`, `DZ`, `F`, `IA`, `NP`, `W`, and `WF`.
`RequirementsAgent`	`ParsedTranscript` to `AgentVerdict`.	Checks GPA, language requirement, elective groups, legacy curriculum markers, and transition rules that require extra INF/social electives or INF321.
`MasterAgent`	Raw transcript and language parameter to final analysis dictionary.	Orchestrates parsing, verification, cross-examination, master aggregation, logging, report generation, tool calling, and final result formatting.
`MasterReportAgent`	Structured analysis summary to JSON report.	LLM role created by `REPORT_SYSTEM_PROMPT`; writes the concise final advisor-style status report. It may use the history tool.
`ReportCriticAgent`	Draft report and analysis data to revised JSON report.	LLM role created by `REPORT_REVIEW_SYSTEM_PROMPT`; reviews the report for factual consistency and removes internal machine values or raw boolean wording.
`check_student_history` tool	Student number/name to JSON history result.	Queries SQLite for the student's most recent prior analysis and returns prior GPA and previous graduation status when found.

Table 2: Agent responsibilities in the implemented system.

7 Algorithmic Approach

The project does not use a single classical machine-learning algorithm for the graduation decision. Instead, it uses a hybrid agentic rule-based pipeline. The algorithm starts by converting raw transcript text into a structured `ParsedTranscript` object with an LLM-backed parser. Then independent rule-verifier agents evaluate different parts of the graduation requirements: mandatory course completion, passed ECTS, GPA, language requirements, elective groups, and legacy transition conditions.

The main aggregation algorithm is strict logical conjunction. Each verifier returns a verdict with supporting details, and the `MasterAgent` marks a student eligible for graduation only if all verifier agents return a positive verdict. This choice was made because graduation eligibility is a rule-governed academic decision. A probabilistic or majority-vote decision could hide a blocking requirement, while strict aggregation makes every missing condition visible.

This design separates uncertain language understanding from deterministic rule checking. The LLM is used where it is strongest: extracting structured information from flexible transcript text and producing readable reports. The actual academic decision is handled by explicit Python rules, which makes the system easier to test, debug, and explain.

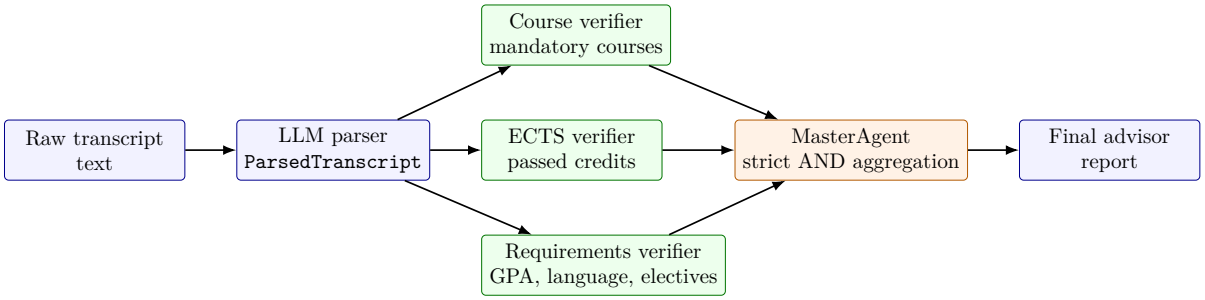

Figure 3: Hybrid agentic rule-based algorithm used for graduation analysis.

8 Decision Logic

The master decision is intentionally strict. The student is marked eligible for graduation only if all verifier agents return a passing verdict:

- all mandatory courses are satisfied by exact course-code matching or official equivalency mappings;
- total passed ECTS is at least 240;
- GPA is at least 2.00;
- language condition is satisfied;
- all required elective groups are satisfied;
- legacy transition rules do not create unresolved extra obligations.

The final output includes both the machine-readable details and the natural language report. The frontend uses this output to show a high-level decision, statistics, agent-level evidence, completed mandatory courses, and missing mandatory courses.

9 Prompts Used

The project uses three explicit system prompts in `backend/agent.py`. For LaTeX portability, arrows and long dashes are print-normalized below; the instruction content matches the implemented prompts.

9.1 Transcript Parser Prompt

```
You are a university transcript parsing expert. Extract ALL courses from a GSU (Galatasaray
↪  University) transcript, including failed ones.

The transcript format is tab-separated with semester headers and course rows:

  2021 - 2022 Güz Yarıyılı
  Kodu    Ders Adı    Notu    Kredi    AKTS    Türü
  INF112-B    Programlamaya Giriş    BA    5.0    6    Z
  ...
  AKADEMİK NOT ORTALAMASI    ANO    KREDİ TOPLAMI    AKTS TOPLAMI
  Yarıyıl    2.10    23.5    30.0
  Genel    2.10    23.5    30.0

Column mapping:
- Kodu -> code (course code, may include section suffix like -A, -B, -C -- keep as-is)
- Ders Adı -> name
- Notu -> grade (AA, BA, BB, CB, CC, DC, DD, F, W, DZ, FF, etc.)
- AKTS -> ects (integer)

GPA extraction:
- The cumulative GPA is on the LAST "Genel" row in the transcript, second column (e.g.
↪  "Genel    2.49    ..." -> gpa = 2.49)

Skip these rows entirely (do NOT extract as courses):
- "AKADEMİK NOT ORTALAMASI ..." header rows
- "Yarıyıl ..." summary rows
- "Genel ..." summary rows
- Semester header rows (e.g. "2021 - 2022 Güz Yarıyılı")
- Column header rows ("Kodu    Ders Adı ...")

Return ONLY valid JSON matching this exact schema:
{
  "student_name": string or null,
  "student_number": string or null,
  "gpa": number,
  "courses": [
    {"code": string, "name": string, "grade": string, "ects": integer}
  ]
}

Rules:
- Include ALL courses even failed ones (F, FF, DZ, W)
- Course codes must be uppercase
- ects must be an integer (use the AKTS column, not Kredi)
- Return raw JSON only, no markdown, no explanation
```

9.2 Master Report Prompt

```
You are the MasterReportAgent for the Computer Engineering department at GSU (Galatasaray
↪  University).

Write a concise 2-3 sentence graduation status report in the target language from the user
↪  payload.
```

```
Base your report strictly on the provided analysis data; do not add or infer anything
↪  beyond what is given.

Use advisor-quality academic prose. Do not expose internal machine values, field names,
↪  enums, or raw booleans in the report text.
Forbidden report wording includes: true, false, pass, fail, is_graduated,
↪  eligible_to_graduate, not_eligible_to_graduate, JSON, boolean.

Interpret the master decision semantically:
- If requirements are satisfied, state that the student meets the graduation requirements.
- If requirements are not satisfied, state that the student does not currently meet the
↪  graduation requirements and summarize the main blockers.

Compare courses strictly by course code. Do not infer missing courses from names. A course
↪  is only missing if its exact code does not appear in the passed courses list.

Return ONLY valid JSON: {"report": "report text here"}
```

9.3 Report Critic Prompt

```
You are the ReportCriticAgent for a university graduation advisor.

Review the draft report against the analysis data and rewrite it if needed.
Preserve the factual decision and all key blockers. Keep it concise, formal, and in the
↪  requested target language.

The final report must not contain internal machine values, field names, enums, or raw
↪  booleans.
Forbidden report wording includes: true, false, pass, fail, is_graduated,
↪  eligible_to_graduate, not_eligible_to_graduate, JSON, boolean.

Return ONLY valid JSON: {"report": "final polished report text here"}
```

9.4 Dynamic Tool-Calling Instruction

During report generation, the master report system prompt is extended with this instruction:

```
You have access to a tool to check the student's history in the database. Use it if a
↪  student number is provided. Incorporate any historical progress into your final report
↪  JSON.
```

The available tool is named `check_student_history`. It accepts `student_number` and `student_name` parameters and returns a JSON status such as `history_found`, `no_history`, or `error`. If history is found, the project also creates a frontend-facing tool log comment comparing previous and current GPA and graduation status.

10 Testing and Evaluation

The repository includes both unit-level and scenario-level validation assets:

- `test_transcript_validation.py` checks single-transcript acceptance and multi-transcript rejection.
- `test_requirements_equivalency.py` checks equivalencies, failed-grade handling, language requirements, elective groups, and legacy transition rules.

- `test_master_report_agent.py` targets report polishing and removal of machine-like boolean language.
- `analysis-input-scenarios/` contains 20 named transcript scenarios covering positive cases, low GPA, ECTS deficit, missing mandatory courses, language failure, modern elective failures, and old-code equivalencies.

The scenario set is especially valuable because it tests the edge cases that make academic advising difficult: legacy curricula, old course codes, replacement rules, and extra elective obligations caused by failed legacy courses.

11 Frontend Interface

The frontend provides the user-facing layer of the Virtual Academic Advisor system. It is implemented with React and Vite, and it communicates with the FastAPI backend through the transcript upload, analysis, result retrieval, and history endpoints. The interface is designed around three main workflows: submitting a transcript for analysis, inspecting a completed graduation report, and reviewing previous analyses.

11.1 Analysis Input Screen

The analysis screen is the entry point of the application. It allows the user to paste an unstructured transcript text or upload a plain-text transcript file. Before sending the transcript to the backend, the frontend performs lightweight validation to detect empty inputs, unsupported file types, and possible multiple transcripts in a single submission. This reduces invalid backend calls and gives the user immediate feedback.

The screen also links the user to the official GSU Computer Engineering curriculum page, making the academic rule source visible from the interface. During analysis, the UI displays progress steps such as transcript upload, parsing, agent evaluation, graduation-condition checking, and report generation. This makes the agentic pipeline visible to the user instead of presenting the system as a black-box response generator.

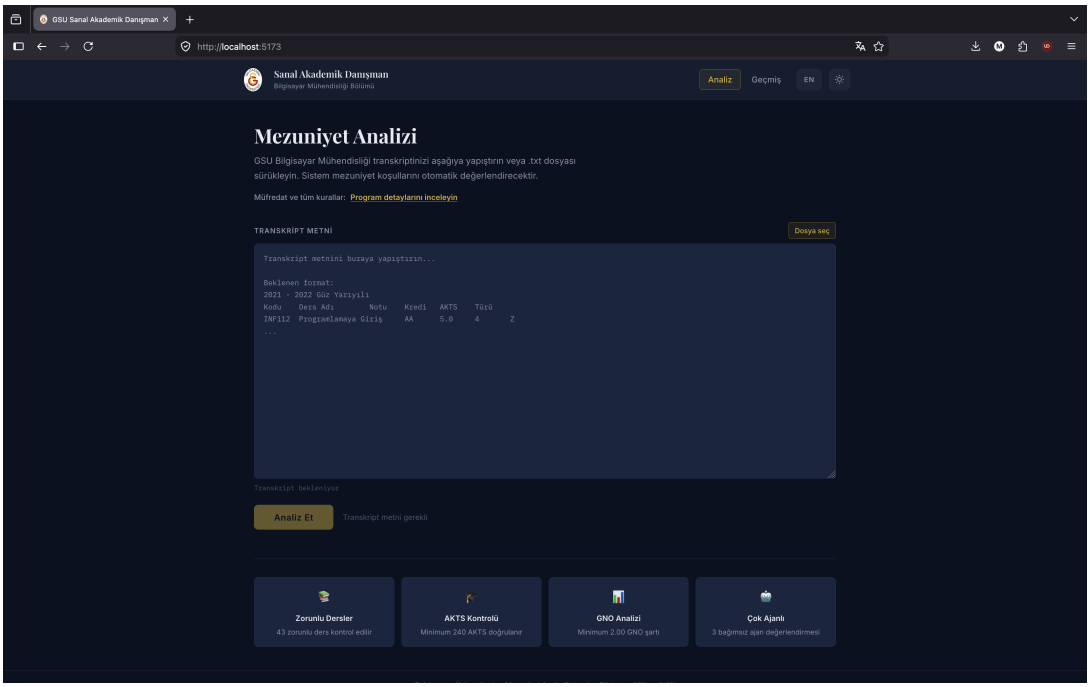

Figure 4: Transcript analysis input screen.

11.2 Analysis Result Screen

The result screen presents the final graduation decision and the evidence behind it. At the top of the page, a status banner indicates whether the student is eligible for graduation or has missing conditions. The LLM-generated advisor report is displayed directly under this status, while structured metrics such as GPA, total ECTS, completed mandatory courses, and missing conditions are shown in separate summary cards.

A key design choice is that the frontend exposes the internal agent verdicts. The user can inspect the outputs of the CourseVerifierAgent, ECTSVerifierAgent, RequirementsAgent, and MasterAgent. This supports transparency: the final decision is not shown as a single unexplained label, but as the result of multiple specialized agent evaluations. The same page also lists completed and missing mandatory courses, allowing the student to identify concrete academic actions needed for graduation.

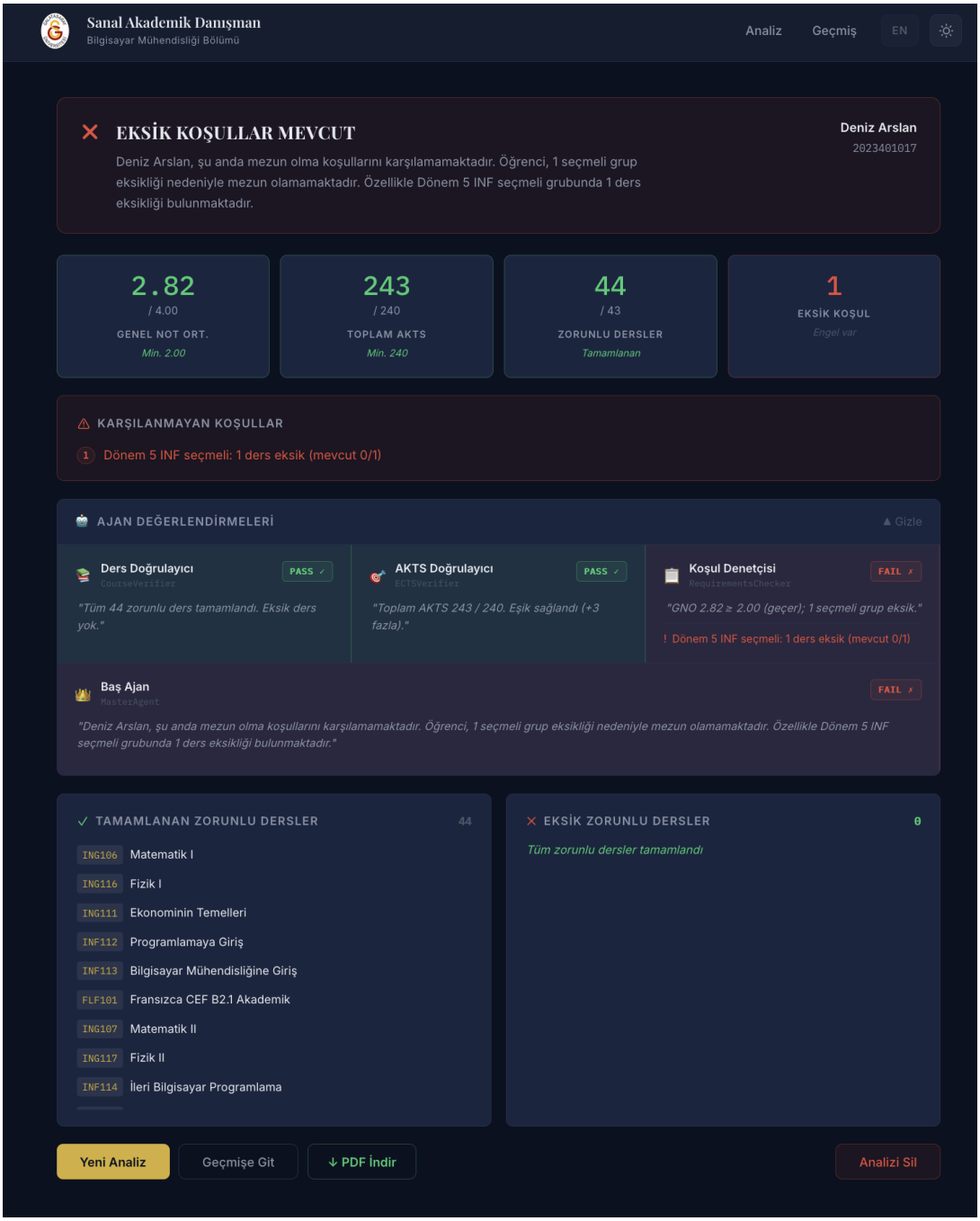

Figure 5: Graduation analysis result screen with agent verdicts and missing conditions.

11.3 Analysis History Screen

The history screen provides persistence at the user-interface level. It lists previously completed analyses with student identity, GPA, ECTS, eligibility status, and analysis date. The user can search by student name or number, filter records by graduation status, sort by date, open a previous result, or delete an analysis.

This screen is also important for the agentic design because the MasterAgent can query historical records through the `check_student_history` tool during report generation. Therefore, the frontend history page is not only a convenience feature; it also reflects the persistent memory layer that can be used by the tool-augmented reporting agent.

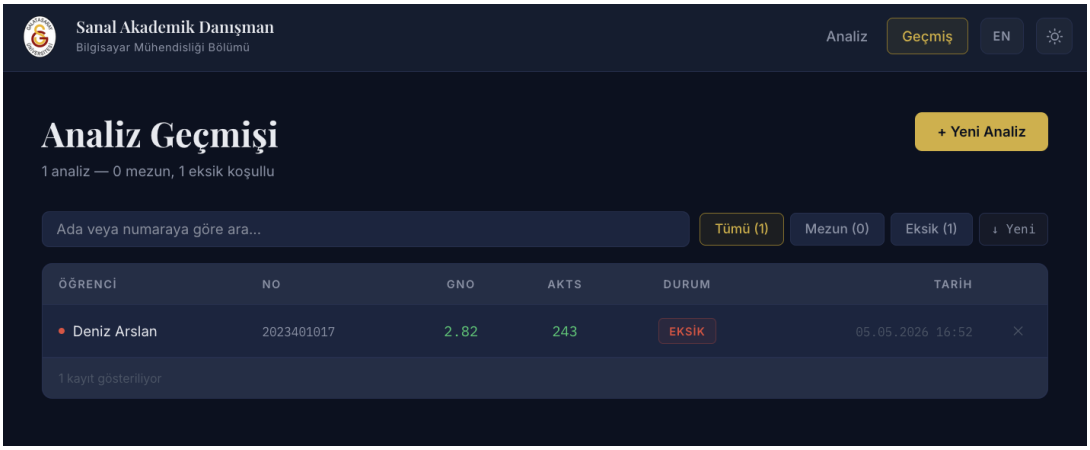

Figure 6: Analysis history screen with search, filtering, and previous results.

12 Strengths

- The final decision is explainable because each verifier produces a verdict, statement, issues list, and details object.
- The LLM parser is schema-constrained with Pydantic validation.
- GPA extraction has a deterministic override based on the last cumulative `Genel` row.
- Course matching is code-based rather than name-based, reducing false matches from translated or abbreviated course names.
- The system handles section suffixes, failed grades, language requirements, elective pools, and old curriculum equivalencies.
- Report generation is reviewed by a second LLM role to improve tone and remove internal machine terminology.
- The UI exposes agent verdicts, making the system more transparent than a single opaque eligibility label.

13 Problems Encountered During Development

Several implementation challenges appeared during the project. The first challenge was transcript structure. Transcript text contains semester headers, course rows, summary rows, cumulative GPA rows, and sometimes repeated or failed courses. Early parsing attempts could confuse summary rows with course rows or extract GPA from the wrong location. To reduce this risk, the project uses a strict parser prompt and a deterministic GPA override based on the last cumulative `Genel` row.

Another difficulty was course matching. The same course can appear with section suffixes such as `-A` or `-B`, and older students may have course codes that map to newer curriculum requirements. The verifier logic therefore normalizes course codes and applies explicit equivalency mappings instead of matching only by course name. This was important because translated or slightly different course names can create false matches.

The legacy curriculum rules also made the decision logic more complex than a simple checklist. Some failed old courses create extra INF or social elective obligations, and failed grades

must not contribute to completed ECTS. These cases required separate test scenarios and verifier details so that the final report could explain not only that a student is blocked, but also why.

Finally, some integration issues appeared between the backend and frontend. For example, the backend requirement list contains 44 mandatory entries while the frontend result card currently displays a hard-coded denominator of 43. This does not affect the backend decision, but it showed the importance of keeping displayed metrics derived from the same source of truth as the verification logic. The input flow also needed validation against multi-transcript submissions, because combining more than one transcript in a single request can produce misleading parsing and analysis results.

14 Limitations and Future Work

- The curriculum is encoded manually. A future version could scrape or import the official ECTS data into a versioned rule database.
- The frontend mandatory-course denominator should be aligned with the backend requirement count.
- The application currently assumes local deployment and does not include authentication or role-based access control.
- Raw transcript text is stored in SQLite; a production deployment should add encryption, retention policy, and stronger privacy controls.
- The LLM parser is robust for the expected transcript format, but additional OCR/PDF parsing would be needed for scanned transcript uploads.
- Prerequisite checking is represented in the official curriculum but is not the central graduation-decision criterion in the current implementation.

15 Conclusion

The project implements a complete Virtual Academic Advisor for GSU Computer Engineering. Its core contribution is a pragmatic multi-agent architecture: LLM-backed agents parse and communicate, rule-verifier agents evaluate academic requirements, and the MasterAgent coordinates evidence, tool use, logging, and final reporting. This design is well suited to academic advising because it keeps high-stakes rule decisions traceable while still benefiting from LLM flexibility for unstructured transcripts and natural-language feedback.

References

[1] Galatasaray University ECTS Information System, *Computer Engineering Undergraduate Program Details*. https://ects.gsu.edu.tr/tr/program/programmedetails/12. Accessed May 9, 2026.